МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

ЛЕКЦІЯ 7. ЗГОРТКОВІ НЕЙРОННІ МЕРЕЖІ ТА КОМП'ЮТЕРНИЙ ЗІР

Львів -- 2026

Лекція 7: Згорткові нейронні мережі та комп'ютерний зір

Вступ: навіщо комп'ютерний зір?

Комп'ютерний зір (Computer Vision) — це галузь штучного інтелекту, мета якої навчити комп'ютер розуміти зображення. Коротко: **дізнатися, що є на картинці, де це знаходиться, і що відбувається**.

Людський зір — це дивовижна система. Ми миттєво розрізняємо тисячі об'єктів, оцінюємо відстань, розпізнаємо обличчя навіть після десятиліть. Для комп'ютера це складне завдання, бо він «бачить» лише набір чисел.

Де використовується комп'ютерний зір сьогодні

- **Робототехніка:** роботи Boston Dynamics орієнтуються в просторі за допомогою камер та CNN
- **Медицина:** системи на основі CNN виявляють рак молочної залози на мамограмах з точністю, що перевищує рівень лікарів-радіологів (дослідження McKinney et al., Nature 2020). Аналогічні системи діагностують рак шкіри, COVID-19 на рентгенах легень
- **Автономне водіння:** камери автомобіля подають зображення в нейромережу, яка приймає рішення про прискорення, гальмування, поворот
- **Доступність:** проєкт Google Guideline використовує комп'ютерний зір, щоб допомогти незрячим людям бігати самостійно — телефон розпізнає розмітку доріжки і подає звукові сигнали
- **Розпізнавання облич:** від розблокування телефону до систем безпеки (з усіма етичними питаннями, які це породжує)

Як комп'ютер «бачить» зображення

Для нас зображення — це картинка. Для комп'ютера — **матриця чисел**.

Цифрове представлення

Кожен піксель чорно-білого зображення — це число від 0 (чорний) до 255 (білий). Зображення розміром, наприклад, 12×10 пікселів — це матриця з 120 чисел. Якщо ви збільшите будь-яке фото достатньо сильно, побачите окремі квадратики-пікселі — кожен з яких має числове значення яскравості.

Для кольорових зображень використовується модель **RGB** — три канали:

- **R (Red):** інтенсивність червоного (0–255)
- **G (Green):** інтенсивність зеленого (0–255)
- **B (Blue):** інтенсивність синього (0–255)

Отже, кольорове зображення розміром 1080×1080 — це тривимірний масив $1080 \times 1080 \times 3$, тобто **3.5 мільйони чисел**. Це і є вхідні дані для нашої нейромережі.

Основні задачі комп'ютерного зору

1. **Класифікація (Classification):** на вході — зображення, на виході — мітка класу. Наприклад, мережа отримує фото президента і повертає ймовірності: "Лінкольн: 0.8, Вашингтон: 0.1, Джефферсон: 0.05...". Це задача **класифікації** — вихід є дискретна категорія. Якщо вихід — неперервне число (наприклад, вік людини на фото), це задача **регресії**.

2. **Детекція об'єктів (Object Detection):** не просто «що?», а «що і де?» — мережа малює прямокутні рамки (bounding boxes) навколо знайдених об'єктів.

3. **Сегментація (Segmentation):** кожному пікселю присвоюється клас — найточніше розуміння сцени.

Проблема ручного виділення ознак

Як відрізнити фото обличчя від фото автомобіля? Інтуїтивний підхід: визначити **ознаки** (features) вручну.

Для обличчя це можуть бути: ніс, очі, рот. Для автомобіля: колеса, номерний знак, фари. Для будинку: двері, вікна, дах.

Цей підхід називається **ручне виділення ознак (manual feature extraction):** людина-експерт визначає, які деталі шукати, програмує детектори цих деталей, а потім класифікує зображення за знайденими ознаками.

Чому це не працює надійно

Реальний світ (і реальні ігри) набагато складніший:

- **Зміна ракурсу (viewpoint variation):** один і той же об'єкт виглядає зовсім інакше спереду, збоку, зверху
- **Зміна масштабу (scale variation):** машина здалеку і вблизи — це різні набори пікселів
- **Деформації (deformation):** тварина може бути в різних позах
- **Перекриття (occlusion):** частина об'єкта закрита іншим об'єктом
- **Зміна освітлення (illumination):** той самий об'єкт при різному освітленні виглядає інакше
- **Фоновий шум (background clutter):** об'єкт зливається з фоном
- **Внутрішньокласова варіація (intra-class variation):** стілець може бути офісним кріслом, табуретом, кріслом-качалкою — але все це «стілець»

> **Висновок:** замість того, щоб вручну визначати ознаки, потрібно **навчити мережу знаходити їх самостійно** із даних.

Ідея: навчити ієрархію ознак автоматично

Дослідження (Lee et al., ICML 2009) показали, що глибокі нейронні мережі природно формують **ієрархію ознак:**

- **Перший шар** виявляє найпростіші візуальні елементи: краї (горизонтальні, вертикальні, діагональні), темні та світлі плями
- **Середні шари** комбінують прості ознаки у складніші: текстури, кути, контури частин об'єктів (очі, вуха, ніс)
- **Глибокі шари** розпізнають цілі об'єкти або їх значущі частини: обличчя, структуру тіла

Це нагадує те, як працює зоровий кортекс людського мозку: перші нейрони реагують на прості стимули (лінії та краї), а глибші — на дедалі складніші патерни.

Питання: яка архітектура мережі найкраще підходить для цієї задачі?

Чому повнозв'язна мережа не підходить для зображень

На попередніх лекціях ми вивчали **повнозв'язні нейронні мережі (fully connected networks)**, де кожен нейрон одного шару з'єднаний з кожним нейроном наступного.

Для зображень це має дві критичні проблеми:

1. Втрата просторової інформації

Щоб подати зображення в повнозв'язну мережу, потрібно «розгорнути» двовимірну матрицю пікселів в одновимірний вектор. При цьому мережа не знає, які пікселі розташовані поряд. Піксель, що сусідить з іншим у зображенні, може опинитися далеко у векторі — і мережа не має способу дізнатися, що вони пов'язані просторово.

2. Вибухова кількість параметрів

Зображення 224×224×3 = 150,528 вхідних значень. Якщо перший прихований шар має 1000 нейронів, це вже **150 мільйонів ваг** лише для першого шару. Тренування стає неефективним, а мережа легко перенавчається.

Ключове питання

> Як використати **просторову структуру** зображення для побудови ефективнішої архітектури?

Відповідь: з'єднувати нейрон не з усіма пікселями, а лише з **маленькою локальною областю** — патчем. Ковзати цим «вікном» по всьому зображенню, використовуючи ті самі ваги. Ця операція і називається **згорткою**.

Операція згортки: покрокове пояснення

Що таке фільтр (ядро згортки)

Фільтр (або kernel) — це маленька матриця ваг, наприклад розміром 3×3. Ці ваги — це параметри, які мережа вивчає під час тренування.

Як працює згортка

Уявіть зображення 5×5 і фільтр 3×3:

1. **Прикладаємо** фільтр до верхнього лівого кута зображення (область 3×3)
2. **Поелементно множимо** відповідні значення фільтра і пікселів
3. **Сумуємо** всі результати — отримуємо одне число
4. **Записуємо** це число в нову матрицю — **карту ознак (feature map)**
5. **Зсуваємо** фільтр на один піксель (або більше — це називається **stride**, крок) і повторюємо

Для зображення 5×5 і фільтра 3×3 з кроком 1 отримаємо карту ознак 3×3.

Що саме шукає фільтр?

Різні набори ваг у фільтрі виявляють різні візуальні ознаки:

- Фільтр з від'ємними значеннями зліва і додатними справа — виявляє **вертикальні краї**
- Фільтр з від'ємними зверху і додатними знизу — **горизонтальні краї**
- Спеціальні комбінації значень — **загострення**, **розмиття**, **виявлення кутів**

Ключова ідея: ми **не програмуємо** ці фільтри вручну. Мережа **сама навчається** підбирати оптимальні ваги для кожного фільтра під час тренування.

Три принципи згорткових шарів

1. **Локальні зв'язки:** кожен нейрон вихідного шару отримує інформацію лише від маленького патча вхідних даних
2. **Множинні фільтри:** різні фільтри шукають різні ознаки — один шукає краї, інший — текстури, третій — кути
3. **Спільні параметри:** той самий фільтр застосовується до кожної позиції зображення, що різко зменшує кількість параметрів

Архітектура згорткової нейронної мережі

CNN — це послідовність блоків, кожен з яких виконує три операції:

1. Згортка (Convolution)

Застосовує набір фільтрів до вхідних даних і генерує **карти ознак**. Кожен фільтр створює одну карту ознак. Якщо фільтрів 32, то на виході — 32 карти ознак.

Формальний запис нейрона згорткового шару для фільтра розміру $\times$:

$$\text{output}(\ ,\ ) = \underset{=1}{\ } \underset{=1}{\ } \cdot \ _{+\ ,\ +} +$$

де — ваги фільтра, — вхідні пікселі, — зсув (bias).

2. Нелінійність (ReLU)

Після кожної згортки застосовується функція активації **ReLU** (Rectified Linear Unit):

$$(\ ) = \max(0,\ )$$

ReLU замінює всі від'ємні значення на нуль. Це проста, але критично важлива операція: без нелінійності послідовність згорток була б еквівалентна одній лінійній трансформації і не могла б моделювати складні залежності.

3. Пулінг (Pooling)

Операція зменшення розміру карти ознак. Найпоширеніший — **Max Pooling:**

- Ділимо карту ознак на блоки (наприклад, 2×2)
- З кожного блоку залишаємо лише **максимальне** значення

Навіщо пулінг:

- Зменшує кількість обчислень у наступних шарах
- Забезпечує часткову **інваріантність до зсувів** — якщо об'єкт зміститься на кілька пікселів, максимум у блоці залишиться тим самим

```
# PyTorch
nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3)
nn.ReLU()
nn.MaxPool2d(kernel_size=2, stride=2)
```

Повна архітектура CNN для класифікації

Класична CNN для класифікації складається з двох частин:

Частина 1: Виділення ознак (Feature Learning)

Послідовність блоків `[Згортка → ReLU → Пулінг]`, що повторюється кілька разів. Кожен наступний блок працює з меншим просторовим розміром, але більшою кількістю каналів (фільтрів).

Частина 2: Класифікація (Classification)

1. **Flatten (розгортання):** перетворення 3D-тензора карт ознак у 1D-вектор
2. **Повнозв'язний шар(и):** звичайна нейромережа, що навчилася на попередніх лекціях
3. **Softmax:** перетворює вихід в ймовірності класів

$$\text{softmax}(\quad) = \frac{\quad}{\quad}$$

Приклад у PyTorch

```
model = nn.Sequential(
    # Блок 1: 3 канали → 32 фільтри
    nn.Conv2d(3, 32, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(2, stride=2),

    # Блок 2: 32 → 64 фільтри
    nn.Conv2d(32, 64, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(2, stride=2),

    # Класифікатор
    nn.Flatten(),
    nn.Linear(64 * 6 * 6, 1024),
    nn.ReLU(),
    nn.Linear(1024, 10),  # 10 класів
)
```

Зверніть увагу: єдине, що змінилося порівняно з попередніми лекціями — це **перша частина** (виділення ознак). Класифікатор у кінці — той самий повнозв'язний шар з softmax, який ми вже знаємо. CNN — це спосіб **автоматично знайти правильні ознаки** для класифікатора.

Що CNN вивчає: ієрархія ознак

Після тренування можна візуалізувати, що виявляє кожен шар мережі:

Шар 1 (Conv Layer 1): найпростіші візуальні елементи

- Горизонтальні, вертикальні та діагональні краї
- Темні та світлі плями
- Прості градієнти кольору

Шар 2 (Conv Layer 2): комбінації простих ознак

- Кути та перетини ліній
- Текстури (повторювані візерунки)
- Контури частин об'єктів (наприклад, округлість ока)

Шар 3+ (глибші шари): семантично значущі частини

- Конкретні частини об'єктів: очі, ніс, вуха (для облич)
- Колеса, фари (для автомобілів)
- Цілі обличчя, структури тіл

Це демонструє головну ідею глибокого навчання: **кожен шар будує представлення на основі попереднього**, створюючи все більш абстрактні та корисні описи вхідних даних.

Застосування CNN: за межами класифікації

Детекція об'єктів (Object Detection)

Задача: не просто визначити клас, а знайти **де саме** на зображенні знаходяться об'єкти та обвести їх рамками.

Наївний підхід: вирізати всі можливі прямокутники з зображення і класифікувати кожен. Проблема: занадто багато варіантів (різні позиції, масштаби, пропорції).

R-CNN (Region-based CNN): спочатку алгоритм пропонує ~2000 «підозрілих» регіонів, потім CNN класифікує кожен. Краще, але все ще повільно.

Faster R-CNN: окрема невелика мережа (Region Proposal Network) навчається сама пропонувати регіони. Зображення проходить через CNN лише **один раз**, потім RPN виділяє регіони з карт ознак. Набагато швидше.

Семантична сегментація

Задача: присвоїти кожному пікселю зображення мітку класу (наприклад: «небо», «дорога», «автомобіль», «дерево»).

Fully Convolutional Network (FCN): мережа, що складається тільки зі згорток (без повнозв'язних шарів). Спочатку зображення стискається (downsampling через згортки і пулінг), потім розгортається назад до початкового розміру (upsampling через транспоновані згортки, `nn.ConvTranspose2d`). На виході — маска з класом для кожного пікселя.

Автономна навігація

CNN приймає зображення з камери автомобіля та, опціонально, грубу карту (GPS), а на виході генерує **команди керування**: кут повороту руля, швидкість. Модель тренується end-to-end — від пікселів до дій — без ручного програмування правил.

Ігрові застосування CNN

CNN знаходять практичне застосування в розробці ігор:

Класифікація ігрових ассетів

Автоматичне сортування текстур, спрайтів, 3D-моделей за стилем, типом, якістю. Наприклад: «це текстура каменю, дерева чи металу?»

QA та тестування

CNN може аналізувати скріншоти гри і виявляти візуальні баги: артефакти рендерингу, проникнення текстур, зниклі об'єкти. Це дозволяє автоматизувати частину тестування.

Аналіз скріншотів та стилю

Визначення візуального стилю сцени, оцінка естетичної якості процедурно згенерованих рівнів.

Розпізнавання жестів та поз

Для ігор з управлінням через камеру — CNN розпізнають жести та пози гравця і перетворюють їх на ігрові команди.

Підсумок

Ключові ідеї лекції

1. **Зображення — це матриці чисел.** Комп'ютер не «бачить» — він обробляє масиви значень пікселів.
2. **Ручне виділення ознак ненадійне.** Варіації ракурсу, масштабу, освітлення роблять жорстко запрограмовані детектори крихкими.
3. **Згортка — ключова операція.** Фільтр ковзає по зображенню, виявляючи локальні патерни. Три принципи: локальність, спільні ваги, множинні фільтри.
4. **CNN = Згортка + ReLU + Пулінг.** Ці три операції, повторені кілька разів, формують потужний екстрактор ознак.
5. **Ієрархія ознак виникає автоматично.** Ранні шари виявляють краї, середні — текстури та частини, глибокі — цілі об'єкти.
6. **CNN — це не лише класифікація.** Та сама ідея згорток лежить в основі детекції об'єктів (R-CNN, Faster R-CNN), сегментації (FCN), навігації, і багатьох інших задач.

Що далі

На наступній лекції ми детальніше розглянемо сучасні архітектури: YOLO для швидкої детекції, U-Net для сегментації, а також познайомимося з **Vision Transformers** — альтернативним підходом до обробки зображень, що використовує механізм уваги (attention) замість згорток.